Week 1

Algorithms_Data Structures

Exercise 2: E-commerce Platform Search Function

```
//java code
import java.util.Arrays;
import java.util.Scanner;

class Product {
    int productId;
    String productName;
    String category;

    public Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    public String getProductName() {
        return productName;
    }
```

```
    public String toString() {
        return "Product{" +
                "productId=" + productId +
                ", productName='" + productName + '\'' +
                ", category='" + category + '\'' +
                '}';
    }
}

public class EcommercePlatform {

    public static int linearSearch(Product[] products, String productName) {
        for (int i = 0; i < products.length; i++) {
            if (products[i].getProductName().equalsIgnoreCase(productName)) {
                return i;
            }
        }
        return -1;
    }

    public static int binarySearch(Product[] products, String productName) {
        int low = 0;
        int high = products.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;
```

```
        int cmp = products[mid].getProductName()
                .compareToIgnoreCase(productName);

        if (cmp == 0)
            return mid;
        else if (cmp < 0)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    Product[] products = {
            new Product(1, "Laptop", "Electronics"),
            new Product(2, "Smartphone", "Electronics"),
            new Product(3, "Tablet", "Electronics"),
            new Product(4, "Headphones", "Accessories"),
            new Product(5, "Smartwatch", "Accessories")
    };
```

```
Product[] sortedProducts = products.clone();

Arrays.sort(sortedProducts,
      (p1, p2) -> p1.getProductName()
            .compareToIgnoreCase(p2.getProductName()));

System.out.print("Enter product name: ");
String name = sc.nextLine();

int linear = linearSearch(products, name);

if (linear != -1)
   System.out.println("Linear Search Found: "
         + products[linear]);
else
   System.out.println("Product Not Found");

int binary = binarySearch(sortedProducts, name);

if (binary != -1)
   System.out.println("Binary Search Found: "
         + sortedProducts[binary]);
else
   System.out.println("Product Not Found");

sc.close();
```

```
    }
}
```

```
Enter product name: Laptop
Linear Search Found: Product{productId=1, productName='Laptop', category='Electronics'}
Binary Search Found: Product{productId=1, productName='Laptop', category='Electronics'}


...Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 7: Financial Forecasting

```
public class FinancialForecast {


    public static double futureValue(double principal,
                                     double rate,
                                     int years) {


        if (years == 0) {
            return principal;
        }


        return futureValue(principal, rate, years - 1)
                * (1 + rate);
    }


    public static void main(String[] args) {


        double principal = 10000;
```

```
        double annualRate = 0.05;
        int years = 5;

        double forecastedValue =
            futureValue(principal, annualRate, years);

        System.out.printf(
            "Forecasted value after %d years: %.2f",
            years,
            forecastedValue);
    }
}
```